
MPMWorlds: Material-Point-Method Simulations for Inferring and Extrapolating Physical Dynamics

Žiga Kovačič
Cornell University

Kevin Ellis
Cornell University

Abstract

To study the ability to infer physical dynamics from videos and extrapolate them forward in time, we assemble a dataset of 2D Material Point Method (MPM) physical simulations covering rich physical phenomena such as deformable objects, fluids, kinetic objects, and emitters. We study code generation and video diffusion approaches on this dataset, identifying their strengths and weaknesses by varying the amount of physically relevant side information. The code generation model, beyond giving a working demonstration of automatic synthesis of MPM simulations, reveals that such an approach struggles with inferring physical parameters from visual input, but relative to video diffusion, produces physically and temporally stable extrapolations forward in time, while the video diffusion model more strongly identifies geometric properties from visual input but produces physically implausible extrapolations. Project page: <https://zzigak.github.io/mpmworlds/>.

1 Introduction

Physical simulation is used across robotics, science and engineering, videogames, cognitive science, and the arts. Up until recently, building a high-fidelity physical simulator handling rich physics, such as deformable objects, fluids, etc., meant painstaking programming using techniques such as Material Point Methods (MPM). But could generative models of source code dispense with the need to hand-program new simulations—and could generative models of videos further dispense with the need to have any simulation code at all? These questions are significant, because authoring precise simulation code is expensive and demands domain expertise, while generative video models might achieve much broader coverage of the everyday world than any symbolic simulation ever could.

The ability to infer the physical dynamics of a scene, and successfully extrapolate to future time points, is also a fundamental test of physical reasoning. While the literature has investigated the ability of generative video models to extrapolate physics forward in time [Kang et al., 2024], such works center on simple rigid body dynamics, not the richer dynamics such as fluids, deformable objects, and various different materials.

To answer these questions, we assemble a new dataset, MPMWorld, of physical simulations covering rich dynamics such as fluids, materials such as snow and sand, deformable bodies, and more (Figure 1). We then study the conditions under which generative models of source code (specifically VLMs) can produce accurate simulation code, given partial information such as a history of past frames, or privileged information such as underlying physical parameters. We also study the conditions under which generative video models (specifically video diffusion models, ‘VDMs’) can render the output of the desired simulation, without ever explicitly constructing the underlying code.

We find both methods achieve only partial success. Code generation excels at consistent long-horizon dynamics, but struggles with geometry. Video diffusion excels at geometry but hallucinates dynamics and deteriorates over long time horizons. We probe the models to understand which elements of the simulations they struggle with, identifying specific classes of materials that are surprisingly challenging (or easy) for each approach.

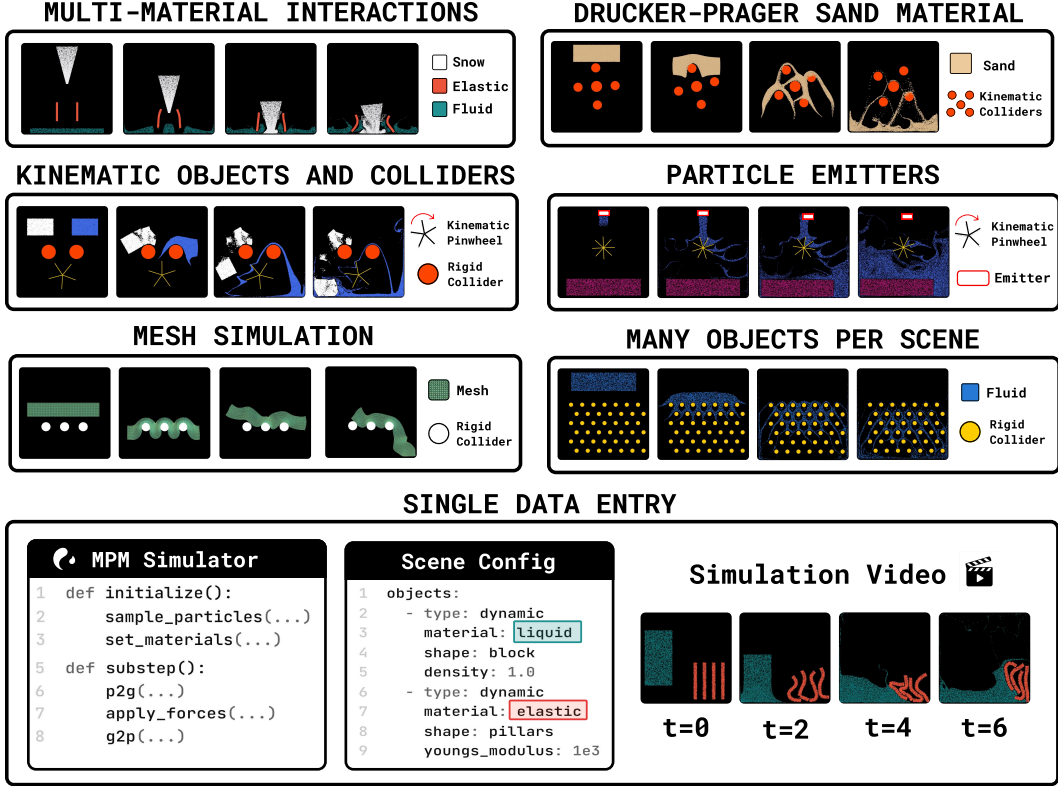


Figure 1: **MPMWorlds dataset overview.** The dataset includes diverse materials and physical interactions. Each entry contains the simulator code, the scene configuration, and the resulting video.

In total we contribute the following:

1. A dataset of 2D physical simulations covering fluids, deformable objects, rigid bodies, emitters, ‘motorized’ objects such as pinwheels and conveyor belts, and more. Each simulation includes both *source code*, *scene configuration file*, and an associated *video of the simulation running*.
2. Neural network models trained on this dataset. We train a variety of models which either generate physics simulation source code, or which generate raw frames, and which condition on a variety of information sources, such as past frames or privileged information such as the numerical values of physical parameters or initial positions.
3. The evaluative role of our dataset and models is to test learning-based methods for inferring physical dynamics, and extrapolating that inference forward in time. Our work analyzes which physical dynamics are challenging to reconstruct and extrapolate. But importantly the analysis offers constructive guidance for future work: By studying which forms of physically-relevant side information most improves the models, we hope to offer constructive guidance on what future modeling should focus on.

1.1 Problem Definition: Inferring and extrapolating physical dynamics.

Given partial information about the physical dynamics of a scene, such as a short video clip, we take *inferring physical dynamics* to mean the recovery (or reconstruction) of a forward simulation of that physical scene. These problems are practically relevant because extrapolations forward in time can serve as a world model for e.g. planning or intuitive physics purposes, and inferring the underlying program is valuable for engineers or artists wishing to work with the physics code. We consider a variety of partial information sources in part because end-users may wish to specify a physical simulation with more than just images or video, but also provide specific physical parameters, while keeping the rest unspecified.

The recovered physical simulation could be represented explicitly as source code, or it could be implicitly represented as neural activations, but in either case, it should support *extrapolation to future time points*, meaning that we can predict what the scene will look like at any point in the future.

We consider scenes generated by physics simulation programs. We write ρ for one such program; different programs simulate different scenes with different objects, materials, etc. A program ρ produces a video v comprising a sequence of frames. Given a prefix of v up to time t , written $v_{\leq t}$, inferring the physical dynamics as a program means to recover a $\hat{\rho}$ equivalent to ρ . Extrapolating to future time points means predicting a video $\hat{v} = \hat{v}_{\leq t} || \hat{v}_{> t}$ (given $v_{\leq t}$).

Given the complexity of physical dynamics that we study, we do not expect inference and extrapolation to be easy. To understand why a model struggles with inference or extrapolation for a particular scene, we can provide physically-relevant *side information*, such as the positions of objects, their densities, etc. We extract different forms of side information from the underlying program, written $f(\rho)$. Models predict either a video or a program, i.e. estimate either $p(v | v_{\leq t}, f(\rho))$ or $p(\rho | v_{\leq t}, f(\rho))$. Figure 2 illustrates the two inference-and-extrapolation pipelines studied in this work: executable simulation synthesis via VLMs and direct pixel-space continuation via VDMs.

2 Related Works

Intuitive Physics and Physical Reasoning. Prior work studies physical reasoning as the ability to infer and predict physical dynamics from visual observations. Cognitive theories propose that human intuitive physics operates through approximate internal simulation [Ullman et al., 2017], while benchmarks such as Physion [Bear et al., 2021] and Physion++ [Tung et al., 2023] evaluate physical understanding in controlled environments. Learned relational and object-centric dynamics models include Interaction Networks [Battaglia et al., 2016], latent physical-property inference from videos [Wu et al., 2015, 2016], and program-like scene representations [Liu et al., 2019]. Our work instead studies whether modern generative models can recover executable or implicit forward models of complex physical scenes.

Programmatic Physical Reasoning and Simulation Code Generation. Recent work explores executable or symbolic representations for physical reasoning. Scene2Prog [Liu et al., 2019] infers programmatic structure from visual scenes, VisPhyWorld [Liang et al., 2026] studies code-driven physical reconstruction, and PhysCodeBench [Xie et al., 2026] evaluates LLM-based physics simulation synthesis. Our setting instead couples executable MPM programs with rendered rollouts and structured scene configurations, enabling controlled comparison between code-space and pixel-space prediction for deformable multi-material dynamics.

Video Generation Models as World Models. Recent work increasingly views generative video models as implicit world models [Ha and Schmidhuber, 2018, Bruce et al., 2024]. Several benchmarks evaluate physical plausibility in generated videos, including VideoPhy [Bansal et al., 2024, 2025], LikePhys [Yuan et al., 2025], PhyWorldBench [Gu et al., 2025], and analyses of physical-law consistency in video generation [Kang et al., 2024]. Unlike prior work focused primarily on perceptual plausibility or short-horizon events [Qiu et al., 2025, Meng et al., 2024], our setting studies long-horizon continuation of deformable multi-material dynamics with paired executable simulations and structured scene representations.

Physical Reasoning Benchmarks. Prior benchmarks evaluate physical reasoning through mechanics puzzles, causal reasoning, and interactive simulation, including PHYRE [Bakhtin et al., 2019], CLEVRER [Yi et al., 2020], Physion [Bear et al., 2021], Physion++ [Tung et al., 2023], and I-PHYRE [Li et al., 2024]. PHYBench [Qiu et al., 2025] studies formal physics reasoning in language models. In contrast, our dataset evaluates generative physical continuation, where models must extrapolate future dynamics either through executable simulation synthesis or direct video generation.

3 MPMWorlds Dataset

To study physical dynamics inference and extrapolation, we introduce **MPMWorlds**, a simulation-based dataset of physical scenes. Each instance includes (i) executable simulation code, (ii) a structured configuration specifying the scene, and (iii) a rendered video of the resulting dynamics. The scene configuration specifies object types, materials, geometry, initial conditions, and appearance,

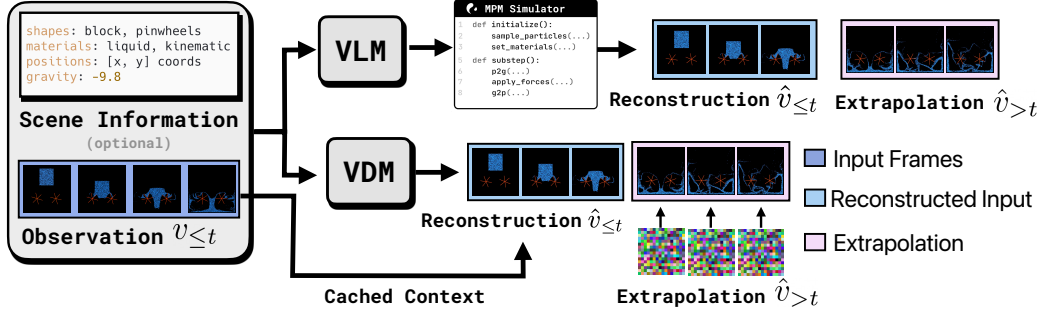


Figure 2: **Reconstruction and extrapolation pipelines.** Models must predict a full video sequence ($\hat{v} = \hat{v}_{\leq t} \parallel \hat{v}_{> t}$) from an initial observation ($v_{\leq t}$). **Top:** The VLM synthesizes a simulation program that executes from $t = 0$, explicitly reconstructing the input (blue) alongside the future extrapolation (pink). **Bottom:** The VDM operates in pixel space, using the input strictly as conditioning to generate future frames.

while the simulation code implements the corresponding physical process. This paired representation enables controlled study of dynamics extrapolation, physical state inference, sensitivity to input modalities, and the role of explicit intermediate representations such as code.

Each configuration is a structured YAML file containing object entries with fields for object type, geometry, material model, initial state, appearance, and optional kinematic or emitter behavior. For example, a dynamic body specifies its shape, color, constitutive model, material parameters, and initial position and velocity; emitters specify spawn region, material, and emission rate.

Dataset Characteristics. We implement the simulations in 2D using the **Material Point Method** (MPM), rendering videos at 512×512 resolution and 30 FPS for 10 seconds. All simulations are written in Taichi, with each program explicitly defining the MPM state update, scene initialization, physical parameters, and time-stepping loop, without relying on high-level simulation APIs.

We use MPM due to its stability and its ability to model a wide range of materials and interactions within a unified framework. It is also widely adopted across computer graphics, visual effects, and engineering communities, making it a practical and well-established choice for simulating diverse physical phenomena. The dataset includes diverse dynamic scenes with deformable and multi-material interactions, including liquids, elastic (particle-based and FEM-based), plastic, snow, sand (including wet interactions), viscoplastic and viscoelastic materials, as well as rigid and kinematic objects [3](#). We also include emitters that continuously introduce material into the scene (e.g., fountains or rain). All scenes are rendered on a high-contrast background to minimize confounding visual factors such as texture or lighting.

3.1 Dataset Creation Pipeline

The dataset is generated using a partially synthetic pipeline combining human-written programs, LLM-based synthetic generation, and data augmentation through small perturbations. It is created entirely from scratch and does not rely on existing datasets or sources. The dataset generation pipeline has a hierarchical structure and we explain the steps below:

Step 1: Seed Simulations. We manually write 35 diverse “seed” simulations with varying object layouts, materials, geometries, initial velocities, kinematic elements (e.g., pistons, gates, rotating objects), and rigid colliders. Simulations are paired with a structured scene YAML config.

Step 2: Base Simulation Generation and Validation. We expand the dataset in the style of Self-Instruct [\[Wang et al., 2023\]](#) by prompting large language models (ChatGPT o3 and Gemini 2.5 Pro) with 2–3 seed examples to generate new simulation programs and corresponding scene configurations, by combining and extending the provided human written code. Generated simulations are executed and filtered using automated checks for runtime failures, trivial outputs, and severe numerical instability. We additionally validate consistency between generated programs and configuration files using an LLM-based verification pass checking object counts, materials, and initial conditions, with flagged failures manually inspected. This process yields approximately 1k valid **Parent** simulations.

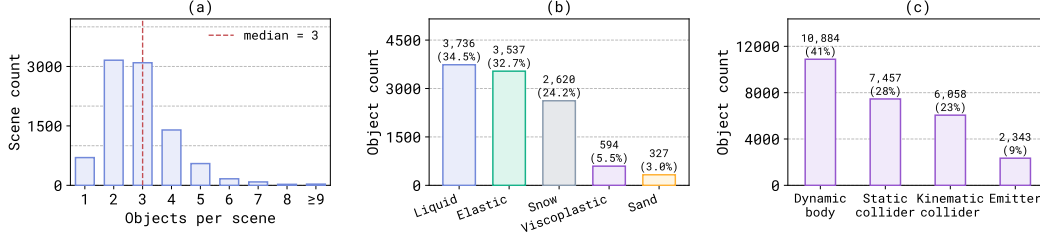


Figure 3: Dataset statistics for **MPMWorlds** across base scenes. (a) Distribution of number of object per scene. (b) Distribution of dynamic-body material types across the dataset. (c) Distribution of object types illustrating the diversity of interaction mechanisms present in the dataset.

We further expand the dataset by prompting the LLM to modify previously generated scenes through changes such as adding or removing objects, altering materials or geometries, and modifying interactions, resulting in approximately 9k **Child** simulations. Together, the Parent and Child simulations constitute the core 10k **Scene Templates** that define the unique physical setups in our dataset.

Step 3: Automated Perturbations. We augment the dataset by perturbing numerical parameters, e.g. object positions, subject to spatial constraints that avoid invalid layouts, such as overlapping objects or objects outside the domain. This reduces bias from LLM-chosen initial configurations, introduces variability in initial conditions, and aids generalization.

3.2 Dataset Statistics

MPMWorlds contains 95,805 rendered simulations, including 9,204 Scene Templates (comprising the full set of Parent and Child simulations) and associated perturbations. Base scenes define the physical setup, while perturbations vary numerical parameters.

Figure 3 summarizes dataset statistics. Scenes have a median of 3 objects and mean of 2.9 objects, with a tail of more complex interactions involving dynamic bodies, colliders, and emitters. The dataset spans multiple classes of physical materials, including liquids, elastic materials, snow, viscoplastic materials, and sand; liquids and elastic materials are most common, followed by snow, while sand and viscoplastic materials provide additional deformation regimes. Overall, 22.5% of scenes contain at least two distinct material types, and 5.3% contain three or more.

Object types include dynamic deformable bodies, static colliders, kinematic colliders, and emitters, enabling interactions such as moving boundaries, externally driven motion, and continuous material injection, substantially increasing the diversity and difficulty of long-horizon dynamics prediction.

4 Experimental Setup and Evaluation

4.1 Input Conditions

We evaluate each model under different input regimes designed to identify which physical properties prove difficult to infer from visual observations alone and if one type of information is much easier to infer from visual information for one approach (VLM or VDM) than the other.

Frames Only. Models input only a video prefix of approximately 2.5 seconds ($v_{\leq t}$, where $t = 2.5\text{sec}$). This is the minimal-information or vision-only setting, so the models have to infer all the information from vision.

Full Configuration. Models receive the video prefix $v_{\leq t}$ together with the complete scene configuration, including object positions, material types, geometry, initial conditions, and physical parameters. This is the maximal-information setting: $f(\rho)$ gives the whole YAML file.

No Materials. Models receive the video prefix $v_{\leq t}$ and the scene configuration with material identities and material-specific parameters removed. This tests inference of material properties from visual evidence alone. Here $f(\rho)$ returns the YAML but with material properties removed.

No Positions. Models receive the video prefix $v_{\leq t}$ and the scene configuration with explicit positional information removed. This tests whether models can infer object layout and initial state from the observed frames. Here $f(\rho)$ returns the YAML but with initial positions removed.

4.2 Models

We compare two types of models, a code generation model (VLM) which produces executable simulation code against a video diffusion model (VDM) that directly predicts future frames.

Vision-Language Models. VLMs predict future dynamics indirectly by generating executable MPM simulation code. Given prefix frames and optional configuration information, the VLM outputs a full Python (Taichi) simulation program $\hat{\rho}$, which when executed predicts video $\hat{v}$.

We use Qwen2.5-VL-7B-Instruct as the base VLM. At inference time, the model receives sampled frames from the prefix, optional side information, and a lightweight program scaffold containing only boilerplate imports and function signatures. The scaffold fixes non-semantic code structure, such as function names and ordering, while the model generates the simulation logic, object initialization, material parameters, and update rules. This focuses learning on the physically meaningful parts of the program rather than arbitrary formatting choices.

Video Diffusion Models. VDMs predict future dynamics directly in pixel space without constructing an explicit physical representation. We use the 14B LongCat video continuation model as our diffusion baseline. The model is conditioned on the observed video prefix and, when available, the same textual scene information used for the VLM input regimes.

4.3 Training and Inference

For all experiments, we construct a 10k-example subset of the dataset with increased sampling weight on base scenes. We evaluate two held-out regimes enabled by the hierarchical dataset structure. The primary **test split** is partitioned at the Parent simulation level: all modifications (Children) and perturbations derived from a held-out base simulation are assigned to test, so models never observe the underlying scene template during training. This evaluates generalization to entirely novel physical scenes.

We additionally construct a **modification validation split** by withholding some modified descendants of training base scenes. This split intentionally preserves partial template overlap while changing object layouts, materials, etc., providing an intermediate regime between exact memorization and fully novel scene generalization.

To improve training and inference efficiency, videos are resized to 256×256 resolution at 15 FPS. For VLM training, we use full supervised finetuning. For VDMs, we observed little difference between full finetuning and LoRA adaptation and therefore use LoRA for all diffusion experiments.

At inference time, we generate multiple candidate programs from the VLM and select among them using evaluation metrics computed on the predicted $\hat{v}_{\leq t}$. Specifically, we sample 15 candidate programs per test example. In contrast, we observe relatively low diversity across diffusion samples under different random seeds and therefore draw a single sample from the VDM. We use 50 denoising steps for diffusion inference, which empirically worked well and which calibrates the inference time compute budget of the two models: 50 denoising steps takes about as long as sampling 15 programs.

4.4 Evaluation Metrics

Evaluating long-horizon physical rollouts requires metrics that capture not only pixel-level similarity, but also temporal consistency, motion fidelity, appearance preservation, and catastrophic physical failures such as collapse or disappearance.

Unlike classical simulation benchmarks, our setting does not provide direct access to particle trajectories or physical state during evaluation. Additionally, many scenes contain highly deformable materials whose shape and topology evolve over time, making standard tracking- or correspondence-based metrics ill-defined.

Prior work on physical video generation primarily evaluates visual plausibility using perceptual or VLM-judged metrics [Bansal et al., 2025, Yuan et al., 2025, Gu et al., 2025]. We evaluate five

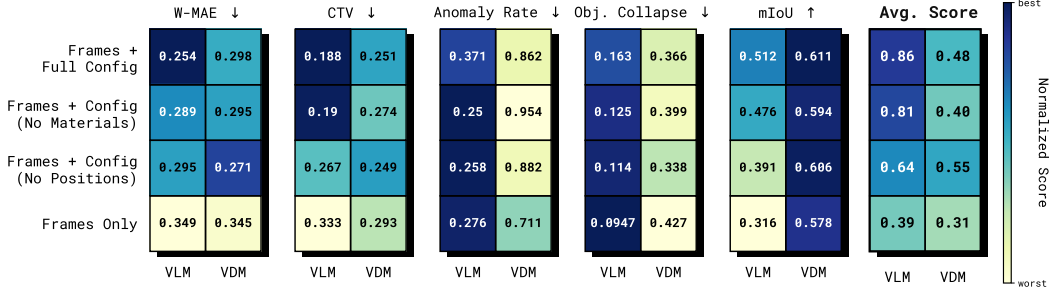


Figure 4: Comparison of VLM- and VDM-based extrapolation across input conditions and evaluation metrics. Values are averaged over the held-out test split. Lower is better for W-MAE, CTV, anomaly rate, and object collapse, while higher is better for mIoU. The final column reports the mean normalized score across metrics for each model and input condition.

complementary aspects of physical continuation: *spatial correspondence*, *appearance and material composition*, *motion fidelity*, *internal physical stability*, and *temporal coherence*. Full details about the metrics are provided in Appendix B.

Mask IoU (mIoU). We measure spatial overlap between predicted and ground-truth foreground occupancy masks. For each frame, RGB images are thresholded into binary foreground masks and the Intersection-over-Union is computed frame-wise.

Object Collapse Score (OCS). We measure prediction-internal temporal stability by checking whether foreground regions generated early in the predicted extrapolation remain visible over time. We compute this by segmenting initial object/material regions, tracking their color-consistent visible area across frames, and measuring the maximum relative area loss.

Windowed Motion Activity Error (W-MAE). We measure whether the prediction exhibits a similar amount of temporal activity as the ground-truth video, independent of exact spatial alignment. Motion activity is computed from frame-to-frame intensity changes aggregated over coarse spatial regions and temporally smoothed using a sliding window. W-MAE is the normalized difference between predicted and ground-truth activity magnitudes over time.

Color Total Variation (CTV). To evaluate appearance and material-composition accuracy, we compare foreground color distributions between predicted and ground-truth rollouts. At each frame, foreground pixels are discretized into RGB histograms and compared using total variation distance.

Temporal Anomaly Rate (RTSJ). We measure temporal coherence by detecting abrupt local appearance changes in predicted rollouts that exceed corresponding changes in the ground truth. We compute spatially aggregated temporal jump magnitudes from local color-histogram changes and flag a rollout as anomalous if its maximum excess jump exceeds a threshold. The reported anomaly rate is the fraction of test samples flagged as anomalous. Full details are provided in Appendix B.3.

Additional Metrics. We additionally report standard pixel-level metrics including MSE and sliced Earth Mover’s Distance (SEMD) in supplementary experiments. However, we do not use them as primary evaluation metrics because they correlate poorly with long-horizon physical consistency.

Metrics enable Best-of- N sampling for program generation. Because we sample multiple programs at test time, the VLM can benefit from Best-of- N sampling: Each program can be executed and its video compared to the ground truth prefix $v_{\leq t}$ under the above metrics, with the best-scoring program being returned. Note that selection only reads the prefix $v_{\leq t}$; the suffix $v_{> t}$ remains hidden from the learner at test time and is used solely to evaluate the selected program.

5 Results and Analysis

Unless otherwise stated, results are on the *fully held-out test split*.

How do executable simulation models compare to direct video prediction? Code generation and direct video prediction exhibit substantially different strengths and failure modes under long-horizon physical extrapolation task (Figure 4, Table 2). Code generation (w/ VLMs) generally

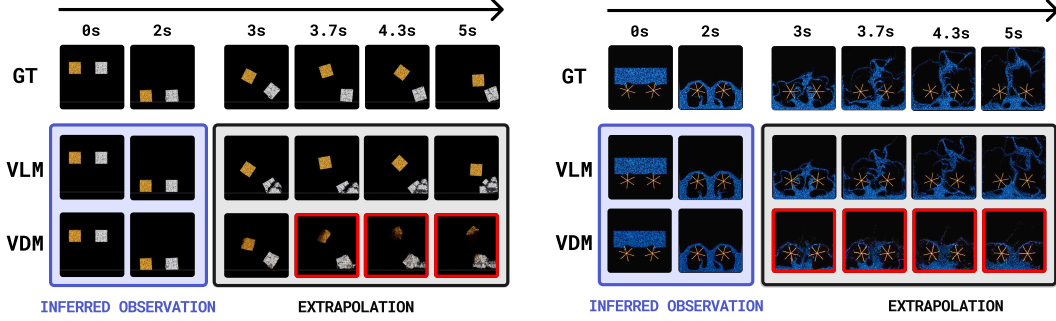


Figure 6: **VDM failure modes in long-horizon physical extrapolation.** **Left:** In an elastoplastic scene, the VLM correctly maintains rigid object permanence and trajectory. The VDM suffers from object collapse, causing the bouncing block to fade and vanish during extrapolation. **Right:** Subjected to high-energy kinematic colliders, the VLM preserves complex fluid volume and splashing dynamics, whereas the VDM prediction fails to capture high frequency motion.

outperforms direct video prediction (w/ VDMs) on most metrics and input conditions, particularly on temporal stability and long-horizon consistency. The largest differences appear in anomaly rate and object collapse, where executable simulations remain substantially more stable over time. Because generated programs are executed inside a physically grounded simulator, successful VLM generations typically preserve object persistence and temporal smoothness unless the predicted code itself becomes numerically unstable. In contrast, VDM predictions frequently exhibit hallucinated motion, disappearing regions, abrupt appearance changes, or temporally inconsistent dynamics, leading to significantly worse anomaly and collapse scores. Representative long-horizon failure cases are shown in Figure 6.

The main exception is mIoU, where VDMs consistently outperform VLMs across all input conditions. This suggests that diffusion models are comparatively better at preserving coarse spatial occupancy and approximate object placement directly from visual observations. However, these spatially plausible extrapolation often deteriorate temporally, producing physically inconsistent motion or unstable long-horizon behavior despite maintaining visually reasonable foreground overlap.

How sensitive are models to structured scene information?

Figure 4 shows that VLM performance is highly sensitive to the availability of structured scene information. Removing material or positional information substantially degrades performance, particularly for appearance fidelity (CTV) and motion consistency (W-MAE). In contrast, VDM performance changes comparatively little across input regimes, suggesting that current video diffusion models make limited use of additional structured physical information even when it is provided explicitly. Instead, they appear to rely primarily on short-term visual extrapolation.

Finally, both model families degrade substantially in the frames-only regime, indicating that long-horizon physical extrapolation remains difficult even when models observe several seconds of rollout history. However, the degradation is qualitatively different: VLM failures typically arise from incorrect physical implementation or unstable generated programs, whereas VDM failures are dominated by accumulated temporal drift, object instability, and incoherent motion over longer horizons. For additional qualitative results, see our [project website](#) and Appendix A.1.

Which materials are hardest to infer from visual information? Figure 5 measures how performance changes when material information is removed from the structured prompt, relative to the full-configuration setting. For VLMs, removing material information has little effect on liquid scenes, but substantially degrades performance for elastic/plastic, snow, and sand scenes. This suggests that fluid-like behavior is either more readily inferred from the visual context or less sensitive to exact

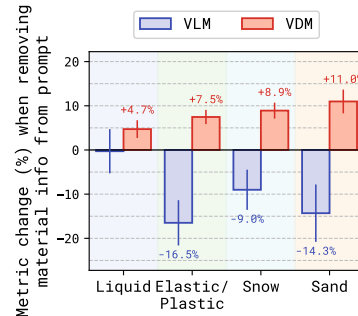


Figure 5: Performance change after removing material information, grouped by material family.

material parameters under our metrics, whereas coherent or history-dependent materials require more explicit material information.

In contrast, VDMs slightly improve when material information is removed across all material categories. This suggests that the VDM relies primarily on visual data and does not consistently benefit from textual material specifications. In some cases, additional structured material information may even act as noise for pixel-space extrapolation rather than improving physical consistency.

Which materials are most challenging for each model class?

Figure 7 aggregates normalized metric performance across tasks for each material family. VLM-based approaches perform strongest on liquid and snow scenes, but struggle substantially on sand dynamics. This is consistent with the fact that sand is implementation wise among the most difficult constitutive models in our dataset, following the approach introduced in [Klár et al., 2016]. Although Drucker–Prager sand is also an elastoplastic material [Li et al., 2026], we separate it from elastic/plastic object scenes because it exhibits qualitatively different dynamics: granular flow, frictional yielding, and loss of coherent object shape. In contrast, our elastic/plastic category primarily contains coherent deformable bodies whose identity and geometry persist over time.

In contrast, VDMs perform comparatively poorly on elastic and plastic scenes. Inspection of these rollouts suggests that the main failure mode is maintaining coherent object trajectories over time. Many elastic scenes contain bouncing or highly dynamic bodies whose motion requires temporally precise geometry and velocity propagation. Video diffusion models frequently produce drift, disappearance, or temporally inconsistent motion in these settings 6. Liquids, while visually complex, are often easier for VDMs because approximate flow structure is sufficient to maintain plausible appearance, even when fine-grained dynamics are inaccurate.

How do failure modes evolve over time? Figure 8 shows the moving-average W-MAE across extrapolation frames, averaged over input conditions. The VDM’s motion-activity error grows quickly after the prefix and remains consistently above the VLM curve, indicating that direct video prediction increasingly deviates from the ground-truth motion magnitude over time. The VLM also degrades with horizon, but more gradually, suggesting that executable simulation provides a more stable long-horizon dynamics prior.

We observe a similar pattern for collapse failures. Figure 13 plots the cumulative fraction of samples that have collapsed by each extrapolation frame. VDM collapses occur early and continue accumulating throughout the rollout, reaching a substantially higher final collapse rate. In contrast, VLM rollouts exhibit far fewer collapses and mostly remain stable after the initial frames.

How well do models perform on modifications of previously observed scenes? Figure 12 compares the fully held-out test split with the modification validation split, where models are evaluated on modified descendants of training scene families. Both model classes improve on the modification split, confirming that fully novel scene templates are harder than new variants of familiar scene structures.

The improvement is larger for VLMs, which gain across nearly all metrics and input regimes, especially in motion fidelity and temporal stability. VDMs also improve, but collapse and anomaly rates remain high, while mIoU remains their strongest metric. Thus, the VLM–VDM gap is not explained solely by memorization or scene novelty, suggesting that executable simulation generation benefits from familiar interaction structure, whereas pixel-space prediction remains better at geometric overlap but weaker at long-horizon dynamics. Figure 9 highlights these contrasting failure modes:

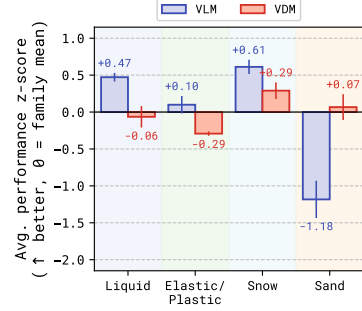


Figure 7: Average normalized performance by material family. Zero denotes each model-family mean across materials.

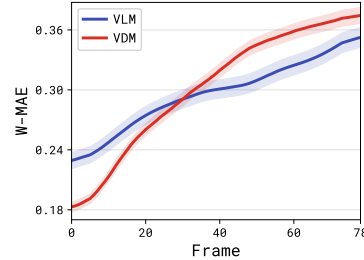


Figure 8: Moving-average W-MAE over continuation frames. VDM motion error grows faster with prediction horizon than VLM error.

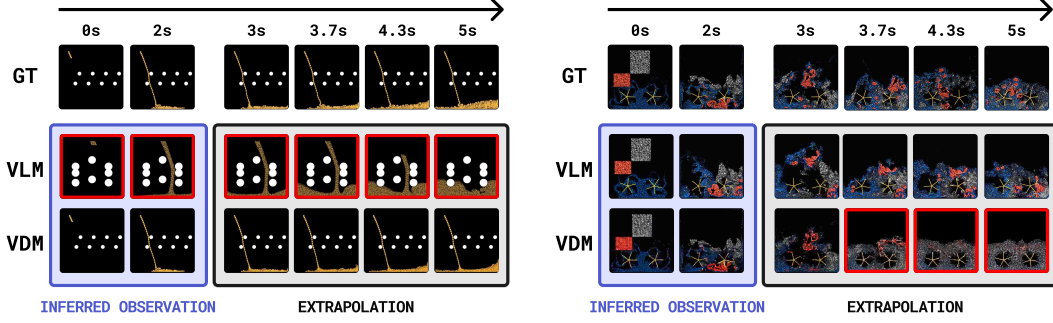


Figure 9: **VLM sensitivity to spatial inputs and compositional scaling.** **Left:** When explicit positional coordinates are withheld, the VLM struggles with visual state estimation, hallucinating an incorrect obstacle layout (red boxes) that alters the physical trajectory. The VDM, relying directly on pixels, preserves the geometry. **Right:** When provided with full configurations, the VLM scales seamlessly to compositional extremes (simultaneous liquid, elastic, and granular materials interacting with kinematic colliders). In contrast, the VDM suffers a catastrophic compositional breakdown, merging distinct materials into a single static average.

VLMs struggle with visual state estimation when positional information is removed, while VDMs fail under compositional multi-material interactions despite preserving coarse geometry.

Can prefix quality predict extrapolation quality? The VLM and VDM exhibit complementary strengths: VLM continuations are typically more physically consistent, while VDM continuations are often geometrically smoother. This motivates a simple question: can observable prefix quality predict which extrapolation to trust?

We test a lightweight prefix-quality gate. For each example, we evaluate how well the VLM reconstructs the visible prefix and use a learned threshold to decide whether to keep the VLM extrapolation or fall back to the VDM prediction. Full details are provided in Appendix D

Table 1 shows that this routing strategy consistently improves over either model alone, particularly for temporal and stability metrics such as W-MAE, anomaly rate, collapse score, and CTV, where the gate routes most samples to the VLM. In contrast, mIoU predominantly routes to the VDM, reflecting its stronger geometric consistency. Overall, these results suggest that prefix reconstruction quality is predictive of long-horizon extrapolation quality and that the two model classes capture complementary aspects of physical extrapolation.

Table 1: VLM vs VDM vs Threshold Gate (top-1) on the **test set**. **Blue** = best, **red** = worst per metric. X% of time the VLM output is selected by the gate.

Input Type	W-MAE ↓			CTV ↓			Anom. ↓			Collapse ↓			mIoU ↑		
	VLM	VDM	GATE	VLM	VDM	GATE	VLM	VDM	GATE	VLM	VDM	GATE	VLM	VDM	GATE
Frames + Full Config	0.254	0.298	0.144 88% LM	0.188	0.251	0.132 82% LM	0.371	0.862	0.001 97% LM	0.163	0.366	0.126 94% LM	0.512	0.611	0.633 23% LM
Frames + Config (No Materials)	0.289	0.295	0.176 80% LM	0.190	0.274	0.139 83% LM	0.250	0.954	0.002 100% LM	0.125	0.399	0.073 100% LM	0.476	0.594	0.629 20% LM
Frames + Config (No Positions)	0.295	0.271	0.197 95% LM	0.267	0.249	0.193 56% LM	0.258	0.882	0.001 100% LM	0.114	0.338	0.063 97% LM	0.391	0.606	0.617 2% LM
Frames Only	0.349	0.345	0.221 91% LM	0.333	0.293	0.213 39% LM	0.276	0.711	0.001 98% LM	0.095	0.427	0.044 95% LM	0.316	0.578	0.617 0% LM

6 Discussion

We introduce a new dataset of physical simulations which help understand the strengths and weaknesses of code generation and video diffusion for physical inference and extrapolation. Despite being limited to 2D physics, the simulations prove challenging for current approaches, owing to the rich variety of physical materials and their interactions instantiated in the data. By feeding privileged information to the models we obtain insight on how to move forward: VLMs benefit strongly from explicit scene information, suggesting they struggle to recover materials, positions, and parameters from vision alone.

Toward Hybrid Architectures. Our analysis of how models leverage side information demonstrates that code generation models (VLMs) excel at long-horizon stability but fail at visual state estimation, struggling to estimate geometry without explicit positional data. VDMs, in contrast, change little when given the same information, suggesting that their main bottleneck is not access to scene state,

but using it to maintain coherent long-horizon dynamics. This points toward hybrid architectures that combine strong visual state estimation with explicit or structured dynamics representations. Indeed, we find that code generation and video diffusion can serve complementary roles for physical prediction, and our simple gating mechanism serves as an effective proof-of-concept for combining their strengths.

Limitations. Our dataset is limited to 2D MPM simulations and does not capture the full material or visual richness of the real world, including materials such as hair, cloth, or highly stiff bodies. Because all videos are generated by ground-truth simulation code, the benchmark may also favor code-generation approaches in ways that may not transfer to natural video. Future work should extend the benchmark to 3D scenes, real-world deformable dynamics, and natural-language side information, making models more useful for artists and everyday users. More broadly, realistic physical video generation may be misused to produce misleading simulations, making controlled evaluation and failure analysis important for responsible deployment.

References

- Anton Bakhtin, Laurens van der Maaten, Justin Johnson, Laura Gustafson, and Ross Girshick. Phyre: A new benchmark for physical reasoning. *arXiv:1908.05656*, 2019.
- Hritik Bansal, Zongyu Lin, Tianyi Xie, Zeshun Zong, Michal Yarom, Yonatan Bitton, Chenfanfu Jiang, Yizhou Sun, Kai-Wei Chang, and Aditya Grover. Videophy: Evaluating physical commonsense for video generation. *arXiv preprint arXiv:2406.03520*, 2024.
- Hritik Bansal, Clark Peng, Yonatan Bitton, Roman Goldenberg, Aditya Grover, and Kai-Wei Chang. Videophy-2: A challenging action-centric physical commonsense evaluation in video generation. *arXiv preprint arXiv:2503.06800*, 2025.
- Peter Battaglia, Razvan Pascanu, Matthew Lai, Danilo Jimenez Rezende, et al. Interaction networks for learning about objects, relations and physics. *Advances in neural information processing systems*, 29, 2016.
- Daniel M Bear, Elias Wang, Damian Mrowca, Felix J Binder, Hsiao-Yu Fish Tung, RT Pramod, Cameron Holdaway, Sirui Tao, Kevin Smith, Fan-Yun Sun, et al. Physion: Evaluating physical prediction from vision in humans and machines. *arXiv preprint arXiv:2106.08261*, 2021.
- Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.
- Jing Gu, Xian Liu, Yu Zeng, Ashwin Nagarajan, Fangrui Zhu, Daniel Hong, Yue Fan, Qianqi Yan, Kaiwen Zhou, Ming-Yu Liu, et al. "phyworldbench": A comprehensive evaluation of physical realism in text-to-video models. *arXiv preprint arXiv:2507.13428*, 2025.
- David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3):440, 2018.
- Bingyi Kang, Yang Yue, Rui Lu, Zhijie Lin, Yang Zhao, Kaixin Wang, Gao Huang, and Jiashi Feng. How far is video generation from world model: A physical law perspective. *arXiv preprint arXiv:2411.02385*, 2024.
- Gergely Klár, Theodore Gast, Andre Pradhana, Chuyuan Fu, Craig Schroeder, Chenfanfu Jiang, and Joseph Teran. Drucker-prager elastoplasticity for sand animation. *ACM Trans. Graph.*, 35(4), July 2016. ISSN 0730-0301. doi: 10.1145/2897824.2925906. URL <https://doi.org/10.1145/2897824.2925906>.
- Minchen Li, Chenfanfu Jiang, Zhaofeng Luo, Wenxin Du, Chang Yu, Žiga Kovačič, and Tianyi Xie. *Physics-Based Simulation*. March 2026. URL <https://phys-sim-book.github.io/>.
- Shiqian Li, Kewen Wu, Chi Zhang, and Yixin Zhu. I-PHYRE: Interactive physical reasoning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=1bbPQShCT2>.

- Jiarong Liang, Max Ku, Ka-Hei Hui, Ping Nie, and Wenhui Chen. Visphyworld: Probing physical reasoning via code-driven video reconstruction, 2026. URL <https://arxiv.org/abs/2602.13294>.
- Yunchao Liu, Zheng Wu, Daniel Ritchie, William T Freeman, Joshua B Tenenbaum, and Jiajun Wu. Learning to describe scenes with programs. In *International conference on learning representations*, 2019.
- Fanqing Meng, Jiaqi Liao, Xinyu Tan, Wenqi Shao, Quanfeng Lu, Kaipeng Zhang, Yu Cheng, Dianqi Li, Yu Qiao, and Ping Luo. Towards world simulator: Crafting physical commonsense-based benchmark for video generation. *arXiv preprint arXiv:2410.05363*, 2024.
- Shi Qiu, Shaoyang Guo, Zhuo-Yang Song, Yunbo Sun, Zeyu Cai, Jiashen Wei, Tianyu Luo, Yixuan Yin, Haoxu Zhang, Yi Hu, et al. Phylabench: Holistic evaluation of physical perception and reasoning in large language models. *arXiv preprint arXiv:2504.16074*, 2025.
- Hsiao-Yu Tung, Mingyu Ding, Zhenfang Chen, Daniel Bear, Chuang Gan, Josh Tenenbaum, Dan Yamins, Judith Fan, and Kevin Smith. Physion++: Evaluating physical scene understanding that requires online inference of different physical properties. *Advances in Neural Information Processing Systems*, 36:67048–67068, 2023.
- Tomer D Ullman, Elizabeth Spelke, Peter Battaglia, and Joshua B Tenenbaum. Mind games: Game engines as an architecture for intuitive physics. *Trends in cognitive sciences*, 21(9):649–665, 2017.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 13484–13508, 2023.
- Jiajun Wu, Ilker Yildirim, Joseph J Lim, Bill Freeman, and Josh Tenenbaum. Galileo: Perceiving physical object properties by integrating a physics engine with deep learning. *Advances in neural information processing systems*, 28, 2015.
- Jiajun Wu, Joseph J Lim, Hongyi Zhang, Joshua B Tenenbaum, and William T Freeman. Physics 101: Learning physical object properties from unlabeled videos. In *BMVC*, volume 2, page 7, 2016.
- Tianyidan Xie, Peiyu Wang, Yuyi Qian, Yuxuan Wang, Rui Ma, Ying Tai, Song Wu, Qian Wang, Lanjun Wang, and Zili Yi. Physcodebench: Benchmarking physics-aware symbolic simulation of 3d scenes via self-corrective multi-agent refinement. *arXiv preprint arXiv:2604.23580*, 2026.
- Kexin Yi, Chuang Gan, Yunzhu Li, Pushmeet Kohli, Jiajun Wu, Antonio Torralba, and Joshua B Tenenbaum. Clevrer: Collision events for video representation and reasoning. In *International Conference on Learning Representations*, 2020.
- Jianhao Yuan, Fabio Pizzati, Francesco Pinto, Lars Kunze, Ivan Laptev, Paul Newman, Philip Torr, and Daniele De Martini. Likephys: Evaluating intuitive physics understanding in video diffusion models via likelihood preference. *arXiv preprint arXiv:2510.11512*, 2025.

Table 2: Visual metrics for shared input regimes across VLM and VDM on **test holdout set** (top-1 per metric, mean $\pm$ 1 SEM). **Blue bold** = best, **red** = worst per metric.

Input Regime	WMAE $\downarrow$	CTV $\downarrow$	Anomaly $\downarrow$	Collapse $\downarrow$	mIoU $\uparrow$	SEMD $\downarrow$	MSE $\downarrow$
VLM	Frames + Full Config	0.254 $\pm$ 0.026	0.188 $\pm$ 0.017	0.371 $\pm$ 0.042	0.163 $\pm$ 0.026	0.512 $\pm$ 0.021	0.064 $\pm$ 0.008
	Frames + No Material Info	0.290 $\pm$ 0.025	0.190 $\pm$ 0.014	0.250 $\pm$ 0.038	0.125 $\pm$ 0.025	0.476 $\pm$ 0.020	0.066 $\pm$ 0.004
	Frames + No Position Info	0.295 $\pm$ 0.023	0.267 $\pm$ 0.014	0.258 $\pm$ 0.040	0.114 $\pm$ 0.024	0.391 $\pm$ 0.016	0.075 $\pm$ 0.004
	Frames Only	0.349 $\pm$ 0.025	0.333 $\pm$ 0.017	0.276 $\pm$ 0.040	0.095 $\pm$ 0.023	0.316 $\pm$ 0.016	0.117 $\pm$ 0.015
VDM	Frames + Full Config	0.298 $\pm$ 0.018	0.251 $\pm$ 0.007	0.862 $\pm$ 0.028	0.366 $\pm$ 0.034	0.611 $\pm$ 0.015	0.044 $\pm$ 0.002
	Frames + No Material Info	0.295 $\pm$ 0.018	0.274 $\pm$ 0.009	0.954 $\pm$ 0.017	0.399 $\pm$ 0.036	0.594 $\pm$ 0.015	0.019 $\pm$ 0.001
	Frames + No Position Info	0.271 $\pm$ 0.017	0.249 $\pm$ 0.007	0.882 $\pm$ 0.026	0.338 $\pm$ 0.032	0.606 $\pm$ 0.015	0.044 $\pm$ 0.002
	Frames Only	0.345 $\pm$ 0.018	0.293 $\pm$ 0.011	0.711 $\pm$ 0.037	0.427 $\pm$ 0.035	0.578 $\pm$ 0.017	0.020 $\pm$ 0.001

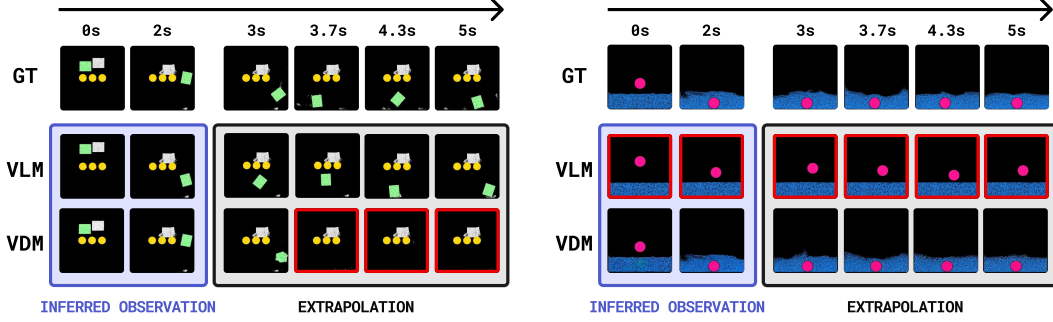


Figure 10: **Failure modes in object permanence and material inference.** **Left:** In an elastoplastic scene, the VLM correctly maintains rigid object trajectories, while the VDM suffers from severe object collapse, causing the bouncing green block to disappear during extrapolation. **Right:** When explicit material properties are withheld from the input prompt, the VLM sometimes struggles with material inference from frames, hallucinating incorrect interaction dynamics (e.g., treating the liquid surface as a rigid boundary). The VDM, relying purely on pixel patterns, more accurately captures object materials.

A Full test set evaluation results

A.1 Example Results

Additional qualitative examples of model failure modes are provided in Figures 10 and 11, illustrating object-collapse failures, material misidentification, and spatial reasoning errors across both model classes.

B Evaluation Metrics

B.1 Mask Intersection-Over-Union

For each frame, RGB images are thresholded into binary foreground masks and the Intersection-over-Union is computed frame-wise.

$$\text{IoU}_t = \frac{|M_t^{\text{GT}} \cap M_t^{\text{pred}}|}{|M_t^{\text{GT}} \cup M_t^{\text{pred}}|},$$

with $\text{IoU}_t = 1$ when both masks are empty. The final score is the mean over all video frames.

B.2 Object Collapse Score (OCS)

We measure prediction-internal temporal object coherence by checking whether objects generated early in the predicted rollout remain visible over time. We segment foreground components in an early frame using brightness thresholding and connected components, merge components with near-identical colors, and use the surviving components as predicted object/material regions. Each region defines an initial area $a_n^{(0)}$ and reference color c_n .

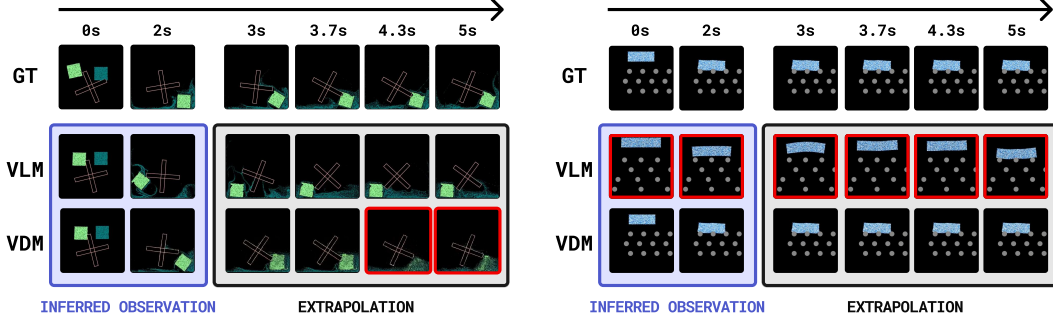


Figure 11: **Contrasting failure modes in complex dynamics and spatial reasoning.** **Left:** In a scene featuring high-energy multi-material interactions (a kinematic pinwheel, fluid, and a rigid block), the VLM accurately preserves object permanence and fluid volume. The VDM suffers from temporal object collapse, causing the rigid block to unphysically dissipate into the surrounding fluid during extrapolation (red boxes). **Right:** When explicit positional coordinates are limited, the VLM struggles with visual state estimation, hallucinating an incorrect geometric layout for the obstacle pegs (red boxes). The VDM, relying directly on pixel-space conditioning, successfully preserves the underlying spatial geometry.

At each later frame t , we estimate the area $a_n^{(t)}$ of region n by counting foreground pixels whose chromaticity and brightness remain close to c_n . The object-level collapse score is

$$s_n^{(t)} = \max \left(0, 1 - \frac{a_n^{(t)}}{a_n^{(0)}} \right),$$

and the per-video score is the worst collapse over all initialized regions and frames:

$$\text{OCS} = \max_{n, t > 0} s_n^{(t)}.$$

Lower values indicate that predicted objects or material regions remain stable; values near one indicate that at least one generated region nearly disappears during the predicted video. For example a value of 0.8 means on averaged over all test samples in that group, the single worst object in each generated video lost 80% of its initial pixel count at some point during the predicted video.

B.3 Temporal Anomaly Rate (RTSJ)

For each frame, we divide the image into coarse spatial regions and compute HSV color histograms within each region. Temporal jumps are measured by comparing histogram statistics across neighboring temporal windows, producing a localized spatiotemporal change signal. We then spatially aggregate this signal to obtain a jump magnitude J_t for each timestep t .

For both the prediction and ground truth, we compute jump magnitudes J_t^{pred} and J_t^{GT} . The excess jump is

$$e_t = \max \left(J_t^{\text{pred}} - J_t^{\text{GT}}, 0 \right).$$

A prediction is flagged as anomalous if

$$\text{anomaly} = 1 \left[\max_t e_t > \tau \right].$$

The reported anomaly rate is the proportion of test samples flagged as anomalous.

C Modification test set values

D Prefix-Quality Threshold Gating

For each metric m and task condition, we compute a prefix score $p_i^m = m(\hat{x}_i^{\text{VLM}}, x_i)$ between the predicted and ground-truth prefix, along with continuation scores $c_{i, \text{VLM}}^m$ and $c_{i, \text{VDM}}^m$ for the corresponding continuations.

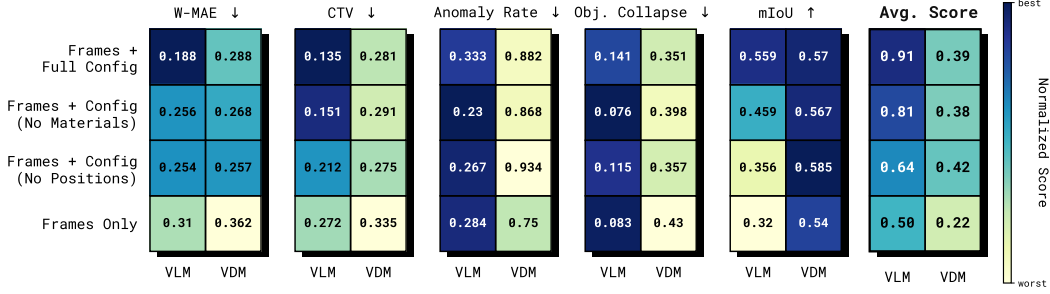


Figure 12: Comparison of VLM- and VDM-based extrapolation across input conditions and evaluation metrics. Values are averaged over the **modification test split**. Lower is better for W-MAE, CTV, anomaly rate, and object collapse, while higher is better for mIoU. The final column reports the mean normalized score across metrics for each model and input condition.

Table 3: Visual metrics for shared input regimes across VLM and VDM on **modification validation set** (top-1 per metric, mean $\pm$ 1 SEM). **Blue bold** = best, **red** = worst per metric.

Input Regime		WMAE ↓	CTV ↓	Anomaly ↓	Collapse ↓	mIoU ↑	SEMD ↓	MSE ↓
VLM	Frames + Full Config	0.188 $\pm$ 0.025	0.135 $\pm$ 0.014	0.333 $\pm$ 0.051	0.141 $\pm$ 0.028	0.559 $\pm$ 0.023	0.053 $\pm$ 0.005	0.018 $\pm$ 0.002
	Frames + No Material Info	0.256 $\pm$ 0.031	0.151 $\pm$ 0.017	0.230 $\pm$ 0.045	0.076 $\pm$ 0.022	0.459 $\pm$ 0.028	0.081 $\pm$ 0.013	0.022 $\pm$ 0.002
	Frames + No Position Info	0.254 $\pm$ 0.027	0.212 $\pm$ 0.017	0.267 $\pm$ 0.048	0.115 $\pm$ 0.029	0.356 $\pm$ 0.023	0.082 $\pm$ 0.012	0.026 $\pm$ 0.002
	Frames Only	0.310 $\pm$ 0.030	0.272 $\pm$ 0.020	0.284 $\pm$ 0.048	0.083 $\pm$ 0.023	0.320 $\pm$ 0.022	0.090 $\pm$ 0.006	0.028 $\pm$ 0.002
VDM	Frames + Full Config	0.288 $\pm$ 0.025	0.281 $\pm$ 0.018	0.882 $\pm$ 0.037	0.351 $\pm$ 0.043	0.570 $\pm$ 0.024	0.075 $\pm$ 0.017	0.015 $\pm$ 0.001
	Frames + No Material Info	0.268 $\pm$ 0.025	0.291 $\pm$ 0.015	0.868 $\pm$ 0.039	0.398 $\pm$ 0.045	0.567 $\pm$ 0.023	0.067 $\pm$ 0.014	0.015 $\pm$ 0.001
	Frames + No Position Info	0.257 $\pm$ 0.023	0.275 $\pm$ 0.016	0.934 $\pm$ 0.029	0.357 $\pm$ 0.043	0.585 $\pm$ 0.023	0.050 $\pm$ 0.005	0.014 $\pm$ 0.001
	Frames Only	0.362 $\pm$ 0.028	0.335 $\pm$ 0.021	0.750 $\pm$ 0.050	0.430 $\pm$ 0.044	0.540 $\pm$ 0.025	0.086 $\pm$ 0.019	0.015 $\pm$ 0.001

We then sweep thresholds τ on a held-out validation set and select the threshold minimizing the average continuation error:

$$\tau^{m*} = \arg \min_{\tau} \frac{1}{N} \sum_{i=1}^N \begin{cases} c_{i,VLM}^m, & p_i^m < \tau, \\ c_{i,VDM}^m, & \text{otherwise,} \end{cases}$$

for lower-is-better metrics. For higher-is-better metrics such as mIoU, the inequality is reversed.

At inference time, we sample multiple VLM candidates, select the candidate with the best prefix score, and apply the learned threshold to decide whether to use the VLM or VDM continuation. Thresholds are learned independently for each metric and task condition.

E Temporal cumulative collapse rate

F Compute Resources.

All VLM and VDM experiments were trained on a server with 4 NVIDIA RTX A6000 GPUs. Training each model required approximately 12 hours. Additional compute was used for dataset generation, rendering, and evaluation.

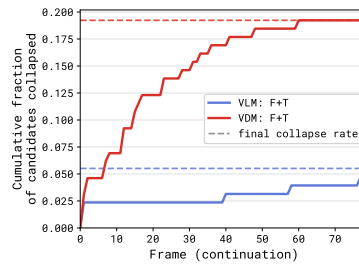


Figure 13: Cumulative collapse rate over extrapolated frames. VDM rollouts collapse earlier and more often than VLM rollouts under the full-configuration setting.